

Bayesian Filtering Approaches for 3D Localization of Quadraped Robots on the Grand Tour Dataset

Group 7

Hao Duan, Xiuping Yang, Zexing Lin, Junxian Wang



introduction



Challenge: State estimation of Quadruped Robots in different complex environments

Equipment: Quadruped Robots

Issues: terrain irregularities, foot slipping, and sensor noise

Dataset: *GrandTour*
ANYmal Grand Tour



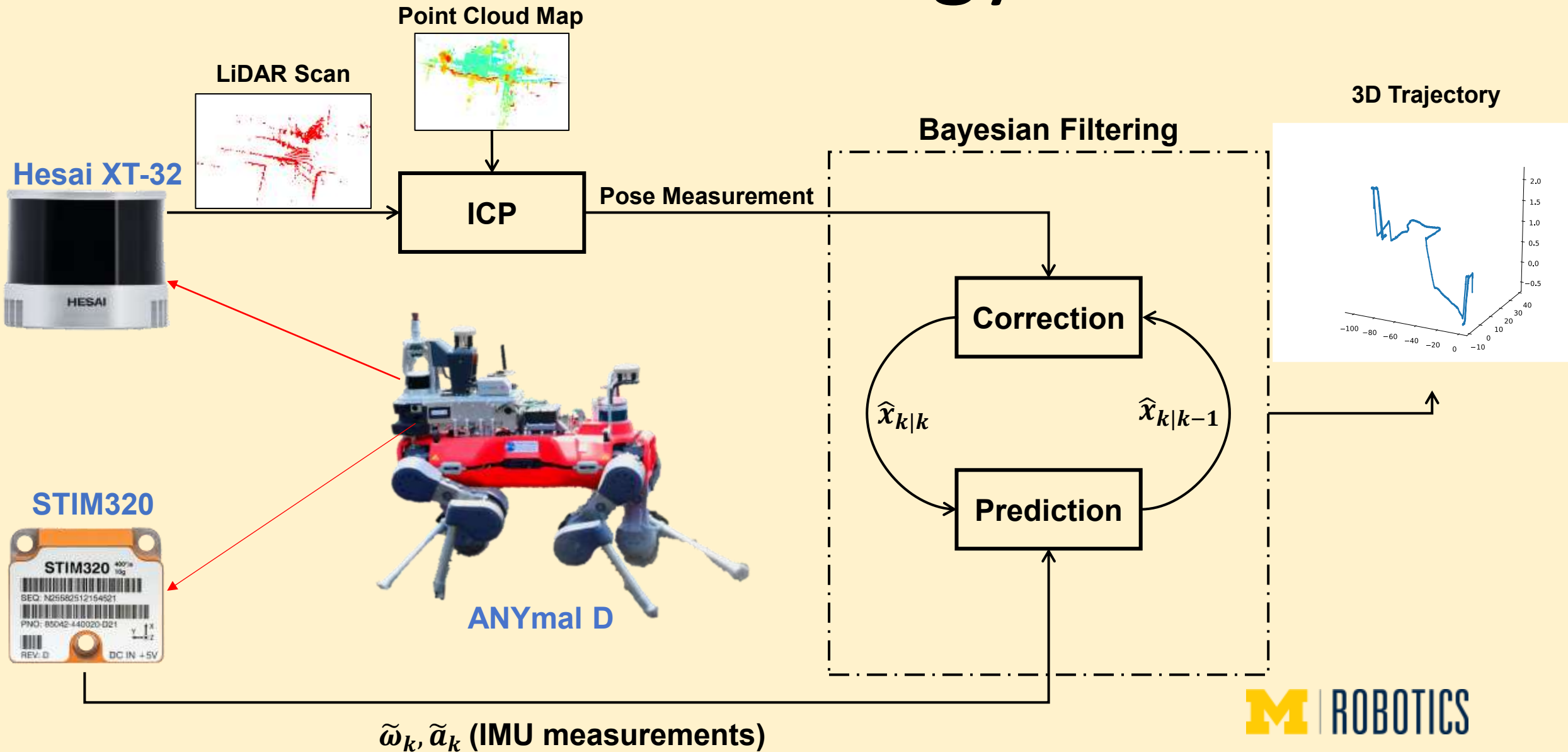
Tested Environments: ETH-1, SPX-1, and SNOW-1

Four Frameworks: EKF, UKF, PF and InEKF

10x Cameras
3x LiDARs
8x IMUs
6x Depth Cameras
12x Actuator
49 Missions
Ground Truth Poses



methodology



Algorithm 1 EKF Algorithm

Require: previous state mean μ_{k-1} , previous covariance Σ_{k-1} , control input u_k , measurement z_k , process noise covariance Q_k , measurement noise covariance R_k , nonlinear state transition $f(\cdot)$, nonlinear measurement $h(\cdot)$

// Prediction (Time Update)

- 1: Predict the state mean
- 2: $\hat{\mu}_k^- \leftarrow f(\mu_{k-1}, u_k)$
- 3: Compute the state transition Jacobian
- 4: $F_k \leftarrow \frac{\partial f(x, u)}{\partial x} \Big|_{x=\mu_{k-1}, u=u_k}$
- 5: Predict the covariance
- 6: $\hat{\Sigma}_k^- \leftarrow F_k \Sigma_{k-1} F_k^T + Q_k$

// Update (Measurement Update)

- 7: Predict the measurement
- 8: $\hat{z}_k \leftarrow h(\hat{\mu}_k^-)$
- 9: Compute the measurement Jacobian
- 10: $H_k \leftarrow \frac{\partial h(x)}{\partial x} \Big|_{x=\hat{\mu}_k^-}$
- 11: Compute the innovation (residual)
- 12: $v_k \leftarrow z_k - \hat{z}_k$
- 13: Compute the innovation covariance
- 14: $S_k \leftarrow H_k \hat{\Sigma}_k^- H_k^T + R_k$
- 15: Compute the Kalman gain
- 16: $K_k \leftarrow \hat{\Sigma}_k^- H_k^T S_k^{-1}$
- 17: Update the state mean
- 18: $\mu_k \leftarrow \hat{\mu}_k^- + K_k v_k$
- 19: Update the covariance
- 20: $\Sigma_k \leftarrow (I - K_k H_k) \hat{\Sigma}_k^- (I - K_k H_k)^T + K_k R_k K_k^T$
(or) $\Sigma_k \leftarrow (I - K_k H_k) \hat{\Sigma}_k^-$
- 22: **return** μ_k, Σ_k

EKF

EKF Algorithm

EKF performs state prediction and measurement update for nonlinear systems.

It linearizes the motion and observation models using Jacobians, which makes it computationally efficient but less accurate under strong nonlinearity.

Algorithm 1 RIEKF (Right-Invariant Error EKF)

Require: previous estimate $\hat{R}_{k-1}, p_{k-1}, v_{k-1}, b_{g,k-1}, b_{a,k-1}$,
previous covariance P_{k-1} , IMU angular velocity ω_k ,
IMU linear acceleration a_k , measurement z_k (e.g., radar position),
time step Δt

- 1: Estimate gyro bias, initial gravity direction, and residual acceleration bias from the initial IMU segment
 - 2: $\tilde{\omega}_k \leftarrow \omega_k - b_{g,k-1}, \quad \tilde{a}_k \leftarrow a_k - b_{a,k-1}$
 - 3: $R_k \leftarrow R_{k-1} \exp((\tilde{\omega}_k)^\wedge \Delta t)$
 - 4: $a_k^w \leftarrow R_k \tilde{a}_k - g$
 - 5: $v_k \leftarrow v_{k-1} + a_k^w \Delta t$
 - 6: $p_k \leftarrow p_{k-1} + v_{k-1} \Delta t + \frac{1}{2} a_k^w \Delta t^2$
 - 7: $\Phi_k, Q_k \leftarrow$ from RIEKF error dynamics
 - 8: $P_k \leftarrow \Phi_k P_{k-1} \Phi_k^T + Q_k$
 - 9: $p_k^{r,-} \leftarrow R_k p_k + p_r$
 - 10: $\tilde{z}_k \leftarrow z_k - p_k^{r,-}$
 - 11: $H_k \leftarrow \left. \frac{\partial h(\cdot)}{\partial \delta \xi^R} \right|_{\hat{X}_k^-}$
 - 12: $S_k \leftarrow H_k P_k H_k^T + R_k$
 - 13: $K_k \leftarrow P_k H_k^T S_k^{-1}$
 - 14: $\delta \hat{\xi}_k^R \leftarrow K_k \tilde{z}_k$
 - 15: $\hat{R}_k \leftarrow \hat{R}_k^- \exp((\delta \hat{\theta}_k)^\wedge)$
 - 16: $p_k \leftarrow p_k^- + \delta \hat{p}_k$
 - 17: $v_k \leftarrow v_k^- + \delta \hat{v}_k$
 - 18: $b_{g,k} \leftarrow b_{g,k}^- + \delta \hat{b}_{g,k}$
 - 19: $b_{a,k} \leftarrow b_{a,k}^- + \delta \hat{b}_{a,k}$
 - 20: $P_k \leftarrow (I - K_k H_k) P_k (I - K_k H_k)^T + K_k R_k K_k^T$
 - 21: **return** $\hat{R}_k, p_k, v_k, b_{g,k}, b_{a,k}, P_k$
-

In-EKF

InEKF Algorithm

InEKF combines IMU-based prediction with radar-based correction for state estimation. By formulating the error on Lie groups, it achieves better consistency and robustness than the standard EKF.

Algorithm 1 UKF-Based Radar-IMU Trajectory Fusion

Require: previous state mean μ_{k-1} , previous covariance Σ_{k-1} , IMU angular velocity ω_k , IMU linear acceleration a_k , radar position measurement z_k

- 1: Estimate gyro bias, initial gravity direction, and residual acceleration bias from the initial IMU segment
- 2: Integrate ω_k to update the sensor orientation
- 3: Transform a_k from the body frame to the world frame and remove gravity
- 4: $\mathcal{X}_{k-1} \leftarrow$ generate sigma points from μ_{k-1} and Σ_{k-1}
- 5: $w \leftarrow$ compute the corresponding sigma-point weights
- 6: **for** each sigma point $\mathcal{X}_{k-1}^{(i)}$ **do**
- 7: $\mathbf{x}_k^{(i)} \leftarrow f(\mathcal{X}_{k-1}^{(i)}, a_k, \Delta t)$
- 8: **end for**
- 9: $\mu_k^- \leftarrow \sum_{i=0}^{2n} w_i \mathbf{x}_k^{(i)}$
- 10: $\Sigma_k^- \leftarrow \sum_{i=0}^{2n} w_i (\mathbf{x}_k^{(i)} - \mu_k^-)(\mathbf{x}_k^{(i)} - \mu_k^-)^T + Q_k$
- 11: Transform the radar ICP trajectory into the IMU frame using the static extrinsic calibration
- 12: $z_k \leftarrow [p_x, p_y, p_z]^T$
- 13: $\mathcal{X}_k^- \leftarrow$ generate sigma points from μ_k^- and Σ_k^-
- 14: $w \leftarrow$ compute the corresponding sigma-point weights
- 15: **for** each predicted sigma point $\mathcal{X}_k^{-(i)}$ **do**
- 16: $\mathbf{z}_k^{(i)} \leftarrow h(\mathcal{X}_k^{-(i)})$
- 17: **end for**
- 18: $\hat{z}_k \leftarrow \sum_{i=0}^{2n} w_i \mathbf{z}_k^{(i)}$
- 19: $\mathbf{v}_k \leftarrow z_k - \hat{z}_k$
- 20: $S_k \leftarrow \sum_{i=0}^{2n} w_i (\mathbf{z}_k^{(i)} - \hat{z}_k)(\mathbf{z}_k^{(i)} - \hat{z}_k)^T + R_k$
- 21: $\Sigma_k^{xz} \leftarrow \sum_{i=0}^{2n} w_i (\mathcal{X}_k^{-(i)} - \mu_k^-)(\mathbf{z}_k^{(i)} - \hat{z}_k)^T$
- 22: $K_k \leftarrow \Sigma_k^{xz} S_k^{-1}$
- 23: $\mu_k \leftarrow \mu_k^- + K_k \mathbf{v}_k$
- 24: $\Sigma_k \leftarrow \Sigma_k^- - K_k S_k K_k^T$
- 25: **return** μ_k, Σ_k

UKF

UKF Algorithm

The algorithm uses IMU data for prediction and radar ICP position measurements for correction.

By propagating sigma points instead of linearizing the model, UKF provides a more accurate estimate under nonlinear dynamics.

Algorithm 1 PF-Based Radar-IMU Trajectory Fusion

Require: Map $\mathcal{M}$, IMU $\mathcal{D}_{imu}$, LiDAR $\mathcal{D}_{lidar}$, Extrinsic $\mathbf{T}_{IL}$ **Ensure:** Trajectory $\mathcal{T}$

```
1: Init: Load map  $\mathcal{M}$ ; Align 1st scan via ICP; Init particles  $\{\mathbf{x}_i, w_i\}_{i=1}^N$ ;  
2: while New LiDAR frame  $\mathbf{P}_k$  arrives do  
3:   1. Prediction (IMU Propagation):  
4:   for each particle  $i$  do  
5:     Integrate IMU  $\mathcal{D}_{imu}$  to update  $(\mathbf{R}_i, \mathbf{v}_i, \mathbf{p}_i)$ ;  
6:     Add process noise  $\boldsymbol{\varepsilon}$ ;  
7:   end for  
8:   2. Multi-Candidate ICP:  
9:   Select  $K$  indices  $\mathcal{J}$  (Best, Mean, Random);  
10:  for each  $j \in \mathcal{J}$  do  
11:     $\mathbf{T}_{init}^{(j)} \leftarrow \mathbf{T}_{WI}^{(j)} \mathbf{T}_{IL}$ ;  
12:     $\mathbf{T}_{icp}^{(j)} \leftarrow \text{ICP}(\mathbf{P}_k, \mathcal{M}, \mathbf{T}_{init}^{(j)})$ ;  
13:     $s_j \leftarrow \text{Fitness} - \lambda \cdot \text{RMSE}$ ;  
14:  end for  
15:   $(j^*, \mathbf{T}^*) \leftarrow \arg \max_j (s_j)$ ;  
16:  3. Update & Gating:  
17:  if  $\text{Fitness}(\mathbf{T}^*) \geq \tau_f$  and  $\text{RMSE}(\mathbf{T}^*) \leq \tau_r$  then  
18:    for each particle  $i$  do  
19:      Update weight  $w_i \propto w_i \cdot \mathcal{N}(\mathbf{x}_i; \mathbf{T}^*, \Sigma)$ ;  
20:    end for  
21:    Normalize  $w_i$ ;  
22:  end if  
23:  4. Estimation & Resample:  
24:  Estimate  $\hat{\mathbf{x}} \leftarrow \sum w_i \mathbf{x}_i$ ;  
25:  if  $N_{eff} < N_{th}$  then  
26:    Resample  $\{\mathbf{x}_i\}$ ; Apply Roughening;  
27:  end if  
28:  Store  $\hat{\mathbf{x}}$  in  $\mathcal{T}$ ;  
29: end while  
30: return  $\mathcal{T}$ 
```

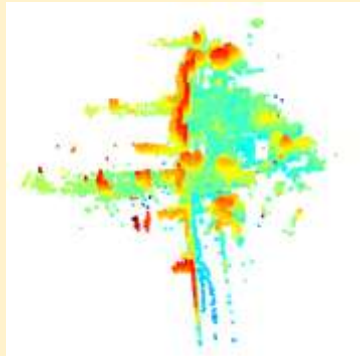
PF

PF Algorithm

The algorithm first propagates particles using IMU measurements and then updates their weights based on ICP matching results. Resampling is applied to keep high-likelihood particles, which improves robustness in complex environments.

Results

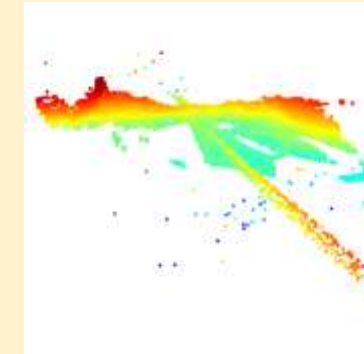
ETH-1



SPX-1



SNOW-1



We evaluate the proposed method on three challenging scenes: ETH-1, SPX-1, and SNOW-1.

The results demonstrate robust performance and consistent geometric reconstruction under diverse environments.

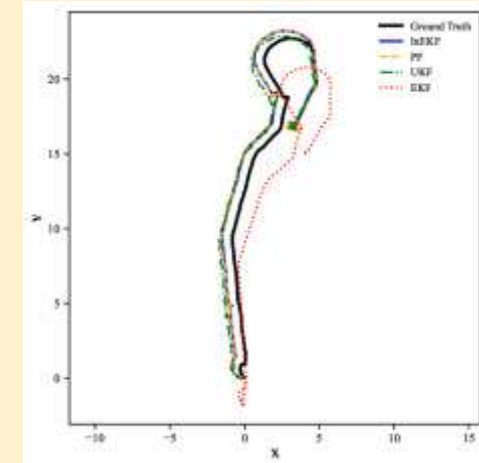
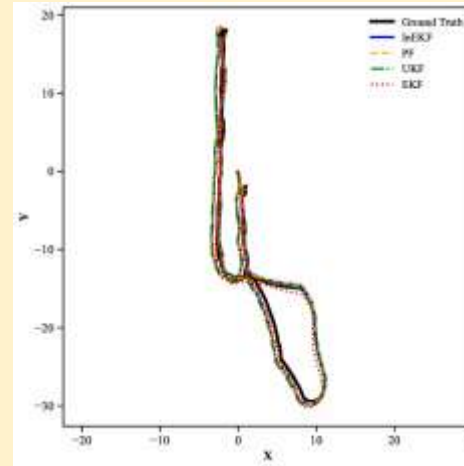
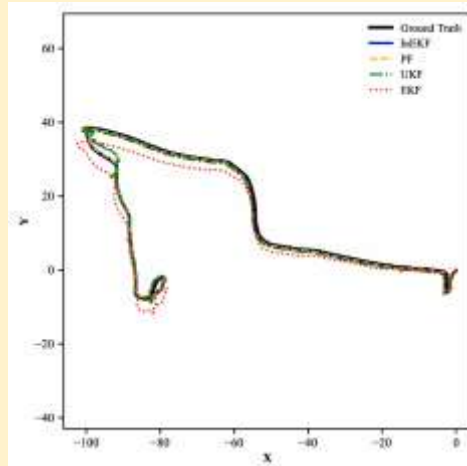
Trajectory comparison

ETH-1

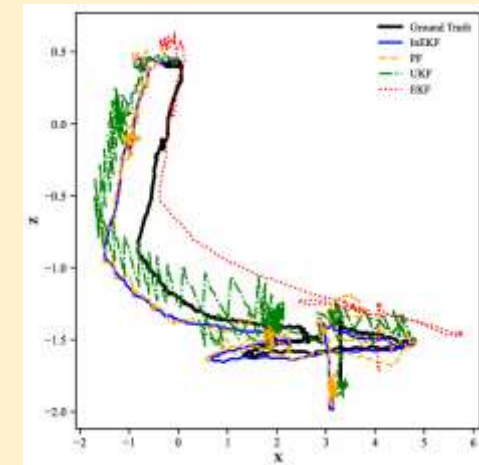
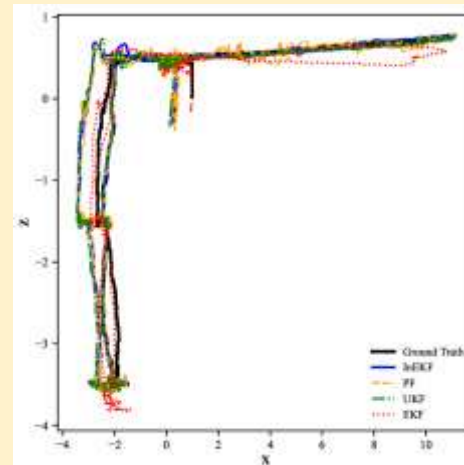
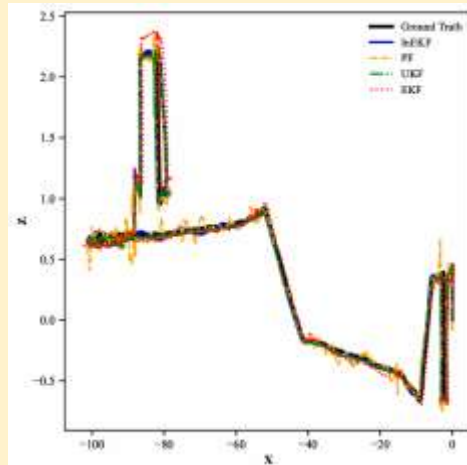
SPX-1

SNOW-1

Top View
(xy)



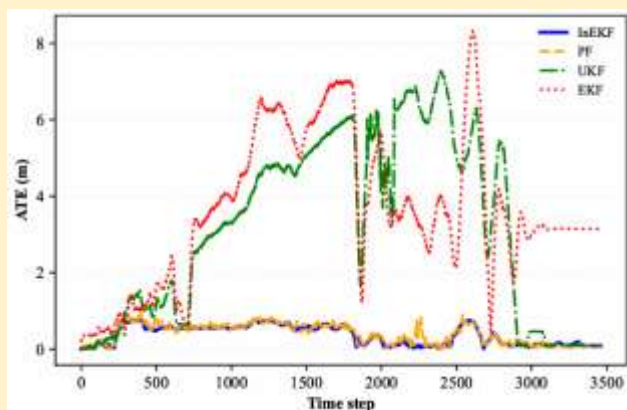
Side View
(xz)



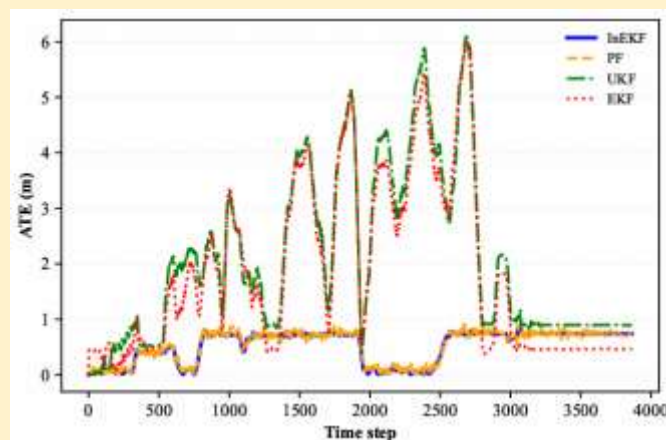
InEKF shows the closest agreement with the ground truth, while EKF exhibits larger deviations, especially in more challenging scenes.

Error & Runtime comparison

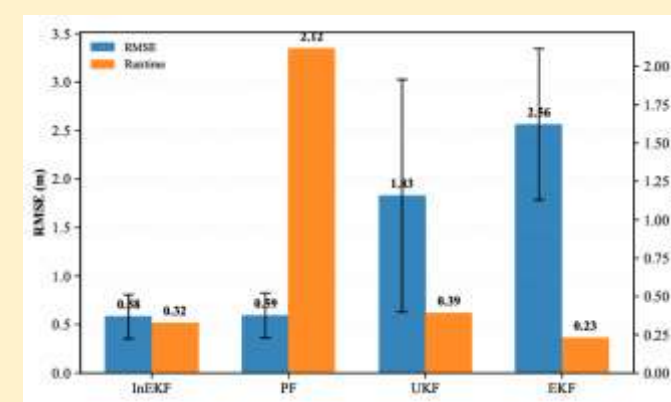
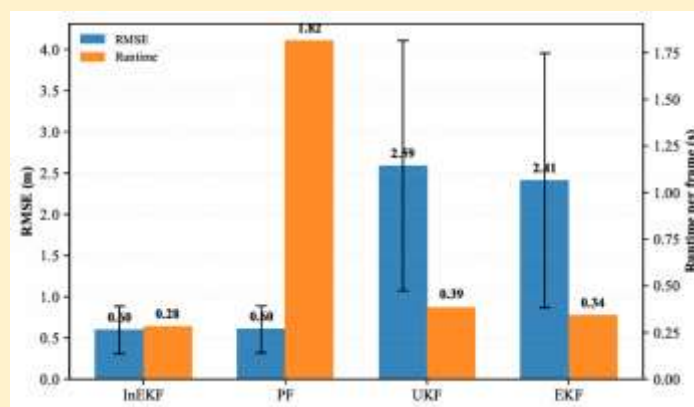
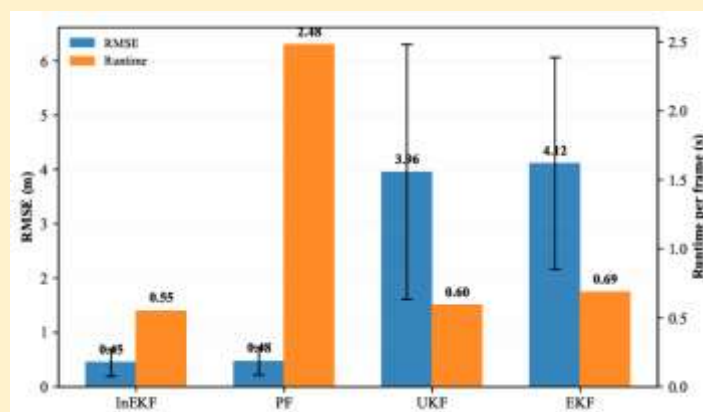
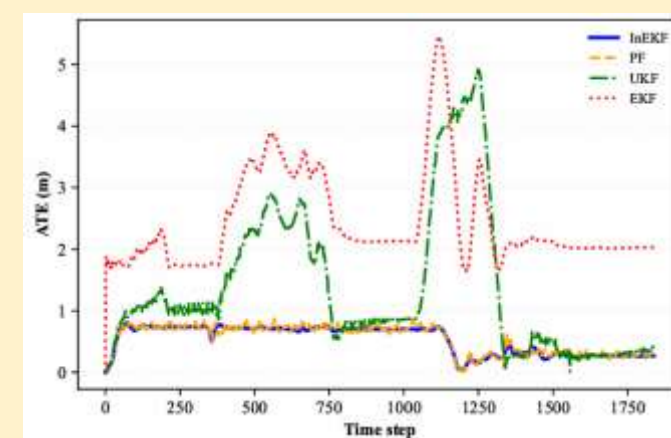
ETH-1



SPX-1



SNOW-1



We further compare the filters in terms of trajectory error and computational cost across the three datasets.

InEKF achieves the best overall trade-off between estimation accuracy and runtime efficiency.

Discussion

1. InEKF performs better in accuracy and computational efficiency
2. PF provides competitive accuracy, but its runtime is much higher.
3. EKF and UKF are more sensitive to nonlinear motion and degraded measurements.

Limitations:

highly dependence on LiDAR & ICP

Future works:

Integrate additional sensing modalities, improve registration strategies, and incorporate contact-aware information.

Thank you!

Group 7